

Assignment 3: Enterprise Multimodal RAG & Microservices

Project Overview

Build a production-grade **Enterprise Knowledge Assistant** capable of ingesting diverse unstructured data (PDFs, PPTs, DOCX, Excel) and answering questions with strict factual accuracy and visual verification.

You will implement a **Decoupled Microservices Architecture**:

1. **Ingestion Service**: A background worker that processes files, manages the "Vision Pipeline," and updates the Vector DB.
2. **Chat Service**: A user-facing API that handles queries, history, and Agentic RAG.
3. **Communication**: The services do **not** talk to each other directly. They share state via **MongoDB Atlas** (Vectors + Chat History).

Key Technical Differentiators:

- **Visual Pipeline (PDF/PPT/DOCX)**: These files are converted to images to support "Visual Citations" (showing the exact page screenshot).
- **Data Pipeline (Excel)**: These are processed using **LlamaCloud** / **Excel Agents** for accurate tabular reasoning *without* screenshots.
- **Tooling**: **LibreOffice** (headless) for conversions in Docker.

Technology Stack

- **Architecture**: Decoupled Microservices
 - **Ingestion Tools**:
 - **PDF/PPT/DOCX**: **LibreOffice** (headless) for conversion to PDF, **PyMuPDF** for screenshotting.
 - **Excel**: **LlamaCloud** (LlamaParse / Excel Agents).
 - **Classification**: Gemini Pro Vision / GPT-4o.
 - **Orchestration**: LlamaIndex.
 - **Database**: MongoDB Atlas (Vector Search + User Chat History).
 - **Backend**: FastAPI.
 - **Frontend**: Custom React/Next.js.
 - **Infrastructure**: Docker, Google Cloud Run.
-

Week 1: Architecture & The Visual Pipeline (Docker Hard Mode)

Learning Objectives

Design a distributed system and master headless document processing in Linux containers.

Tasks

Day 1-2: Microservices & Docker Setup

1. Architecture Design:

- Design the two services (ingestion vs chat).
- Shared State:** Define the MongoDB schema that both services will access.
- Health Check:** Implement a `/health` endpoint in the Chat Service that checks if vectors exist in MongoDB (to know if ingestion happened).

2. Ingestion Dockerfile (The Challenge):

- Create a Dockerfile for the Ingestion Service.
- Requirement:** It must install `libreoffice-headless` (or equivalent Linux package) to enable file conversion.
- Verify:* Run the container locally and successfully convert a dummy `.docx` to `.pdf`.

Day 3-5: The Visual Conversion Pipeline

1. Format Unification:

- Implement logic to detect file type.
- DOCX/PPT:** Run `subprocess.call(['libreoffice', '--headless', '--convert-to', 'pdf', ...])` to create a PDF.
- PDF:** Ready for processing.

2. PyMuPDF Screenshotting:

- Use **PyMuPDF (fitz)** to iterate through every page of the generated PDF.
- Render each page as a high-quality PNG image.
- Store images in Cloud Storage (GCS/S3) and keep the URL.

3. Vision Classification:

- Send the generated page images to **Gemini Pro Vision**.
- Prompt:** "Analyze this page image. Does it contain useful knowledge? Answer YES/NO."
- Filter out blank pages or purely decorative cover slides.

Deliverables

- Architecture Diagram (Decoupled).
- Ingestion Dockerfile with working LibreOffice.
- Script that takes a DOCX/PPT and outputs a folder of "valid" page images.

Week 2: Dual-Path Ingestion (Visual vs. Excel)

Learning Objectives

Handle tabular data correctly using specialized agents and merge all data into a Unified Vector Store.

Tasks

Day 1-3: The Dual Ingestion Paths

1. **Path A: Visual Documents (PDF/PPT/DOCX):**
 - For pages flagged "YES" by the Vision model, send the *image* (or text) to **LlamaParse** to extract Markdown.
 - **Metadata:** You must link the extracted text to the specific screenshot URL.
2. **Path B: Excel Files (LlamaCloud):**
 - **Do not take screenshots.**
 - Use **LlamaCloud** (specialized Excel parser) to extract sheet data as structured text/HTML.
 - **Metadata:** Store `file_name` and `sheet_name`, but leave `screenshot_url` empty or null.

Day 4-5: Vector Storage & Persistence

1. **Unified Ingestion:**
 - Use LlamaIndex `VectorStoreIndex` backed by MongoDB Atlas.
 - Ingest both Document nodes (with visual metadata) and Excel nodes (without) into the *same* index.
2. **Verification:**
 - Query the Vector DB manually: `db.collection.find({"metadata.screenshot_url": {"$exists: true}})` to ensure visual links are saved.

Deliverables

- Dual-path ingestion pipeline.
- MongoDB populated with vectors containing correct metadata.

Week 3: Agentic RAG & Chat Service

Learning Objectives

Build the Chat API with LlamaIndex, handling distinct retrieval strategies and conversation history.

Tasks

Day 1-2: Conversation History

1. Mongo Chat Store:

- Implement **LlamaIndex Memory** modules backed by MongoDB.
- Store every user query and AI response in a `conversations` collection.
- *Constraint:* The Ingestion Service does **not** touch this collection.

Day 3-5: The RAG Agent

1. Citation Logic (Crucial):

- Configure the LlamaIndex Query Engine to return **source nodes**.
- **Response Formatting:** The API must return:
 - The Answer.
 - A list of `source_nodes`.

2. Hybrid Logic:

- The backend logic must handle the mixed sources:
 - If source is PDF/PPT `file_name` Return `screenshot_url`.
 - If source is Excel `file_name` Return `sheet_name` and raw row data.

Deliverables

- Chat Service API (POST `/chat`) returning Answer + Structured Sources.
- MongoDB storing chat history.

Week 4: Frontend & Visual Verification

Learning Objectives

Build a UI that handles the "Hybrid" citation nature (Visual vs. Data).

Tasks

Day 1-3: Chat Interface

1. Basic UI:

- Build a React/Next.js chat window.
- Render the bot's markdown response.

2. Citation Rendering:

- Parse the sources returned by the API.
- Add clickable citation numbers `[1]`, `[2]` in the text.

Day 4-5: The Verification Modal

1. **Visual Modal (PDF/PPT):**
 - On click [1]: Open a modal showing the **Screenshot** fetched from the `screenshot_url`.
 - Allow users to zoom/pan the image.
2. **Excel Fallback:**
 - On click [2] (Excel source): Display a "Data Card" showing the raw row data or a download link for the original file.

Deliverables

- Frontend with "Click-to-Verify" logic.
- Handling of both Image and Text-only citations.

Week 5: Deployment & Integration

Learning Objectives

Deploy the microservices and ensure the pipelines are robust.

Tasks

Day 1-3: Docker Optimization

1. **Ingestion Container:**
 - Optimize the LibreOffice installation (it can be heavy).
2. **Docker Compose:**
 - Orchestrate Ingestion, Chat, and Frontend locally.

Day 4-5: Cloud Run Deployment

1. **Deploy Ingestion Service:**
 - Deploy as a Cloud Run **Job** (ideal for batch processing) or a Service with long timeout.
 - *Note:* Increase memory limits (2GB+) for PDF processing.
2. **Deploy Chat Service:**
 - Deploy as a standard auto-scaling Cloud Run Service.
3. **Final Integration Test:**
 - Upload a messy PPT and a clean Excel file.
 - Ask a question requiring data from both.
 - Verify the response cites the PPT (with image) and the Excel (with data).

Deliverables

- Live URL.
- Dockerfiles handling complex dependencies (LibreOffice/PyMuPDF).
- Demo Video showing the full "Hybrid" RAG flow.

Assessment Criteria

1. **Hybrid Handling:** Does the system correctly take screenshots of PDFs/PPTs but *not* Excel files?
2. **Visual Verification:** Does clicking a citation open the *correct* page image?
3. **LibreOffice Integration:** Does the Docker container successfully convert DOCX/PPT without crashing?
4. **Architecture:** Are the services truly decoupled (connected only via Atlas)?

So i want to prepare a deck for casinos can you find me some suitable content for that

Do we have a content deck for VIX Micro ?

Is there a slide that i can use for Bilingualism & Biculturalism ?